

Introduction to Blockchain and Smart Contracts

Lecture 1: Introduction to Blockchain and the Course

Subodh Sharma | IIT Delhi | Jan 18, 2024

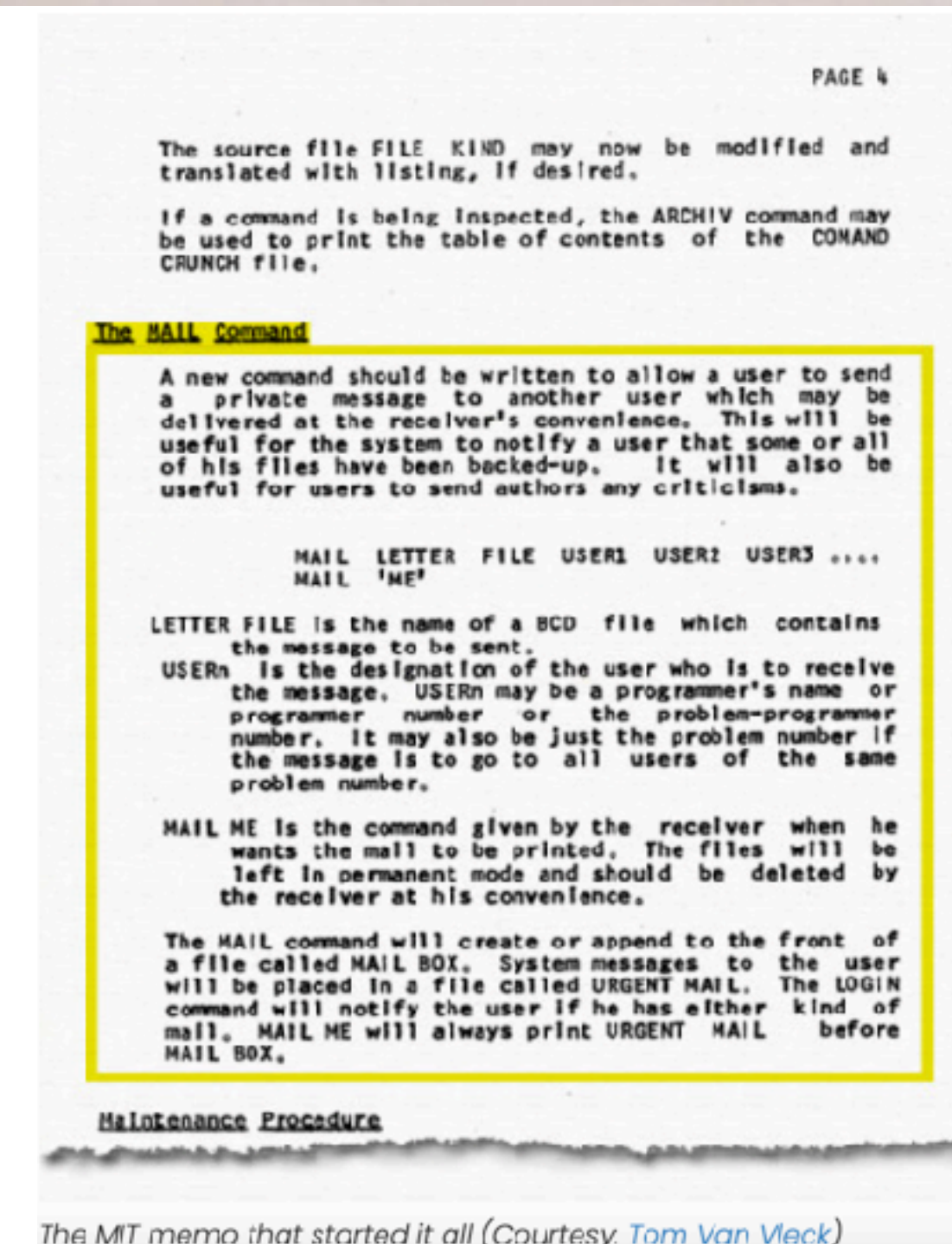


The Past: Internet of Connection (Web 1.0)

- Early Internet — DARPA Project in 1970
 - Through TCP/IP access hypertext documents from any node using TCP/IP
 - Resulting information system — **world wide web!**
- Led to the development of online Applications such as email etc.
- “Read-only” content from central authority pushed to users
- Worth reading: <https://www.emailonacid.com/blog/article/email-marketing/history-of-email/>



Courtesy: Internet





The Present: Internet of Information (Web 2.0)

- User-generated content; **data became a commodity!**
 - Read-write Web!
- Flow of data as information but not **value!**
 - Email a document, audio/video file etc.
 - It is **abundant**, **modifiable**, **perishable!**
- Has it really transformed how we do business?
 - Exist powerful, well-organised and efficient intermediaries (like Banks, think UPI)
 - But operationally **opaque!**
 - Open to internal attacks (think **rogue employees** — PNB bank loans etc.)



Courtesy: Internet



The Future: Internet of **Value** (Web 3.0)

- Instead of flow of information, **flow of value** will take place
 - The speed at which information flows, value will also now be exchanged at the same speed
 - Known as Read-Write-**Own** Web
- Each individual owns his/her data and identity
- Community owned network — thus breaking data silos
- Security is not achieved by intermediaries at the expense of privacy
 - But by sophisticated cryptography!

Decentralisation

Why it is **AND** what it is not!

- **Why do we need it?**
 - **Single point of failure:** outages, censorship, policy changes, insider risk
 - **Single point of control:** gatekeeping, permissioned access, arbitrary reversals
 - **Single point of trust:** you must trust a single institution's correctness + **honesty**
- **Decentralisation $\neq$ No Trust**
 - Replacing **trust in single actor** to a protocol executed by **a set of actors**

Decentralisation in Layers

- **Logical (de) centralization** — do the interface and data structures of the system presents appear to be a single monolithic object, or a distributed object?
- **Architectural (de) centralization** —how many physical computers is a system made up of?
- **Political (de) centralization** —how many individuals/organisations ultimately control those computers that the system is made up of?





Exchange of Value

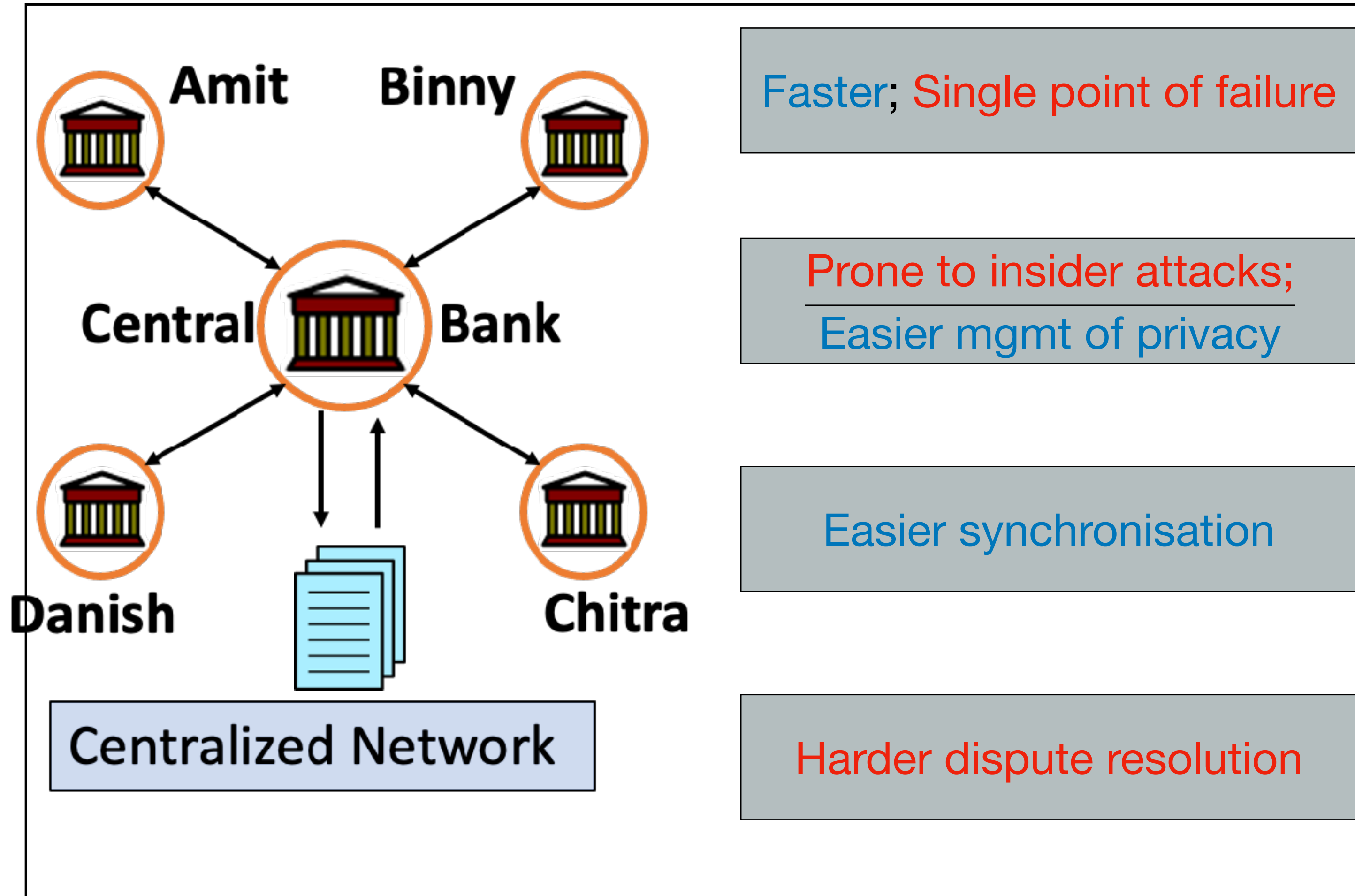
- **Traditional financial arrangement:**
 - Barter system
 - A needs food, B needs medicine; **Expensive coordination!**
 - Problems in scaling up!
 - Credit card system:
 - B gets the medicine, but now owes a debt to A that can be paid later.
 - **Simplicity** — **directly interacting with the seller**; but **privacy harm!**
 - Cash:
 - **No identity requirement**; Stores value of the transacted object; accepted as measure of worth



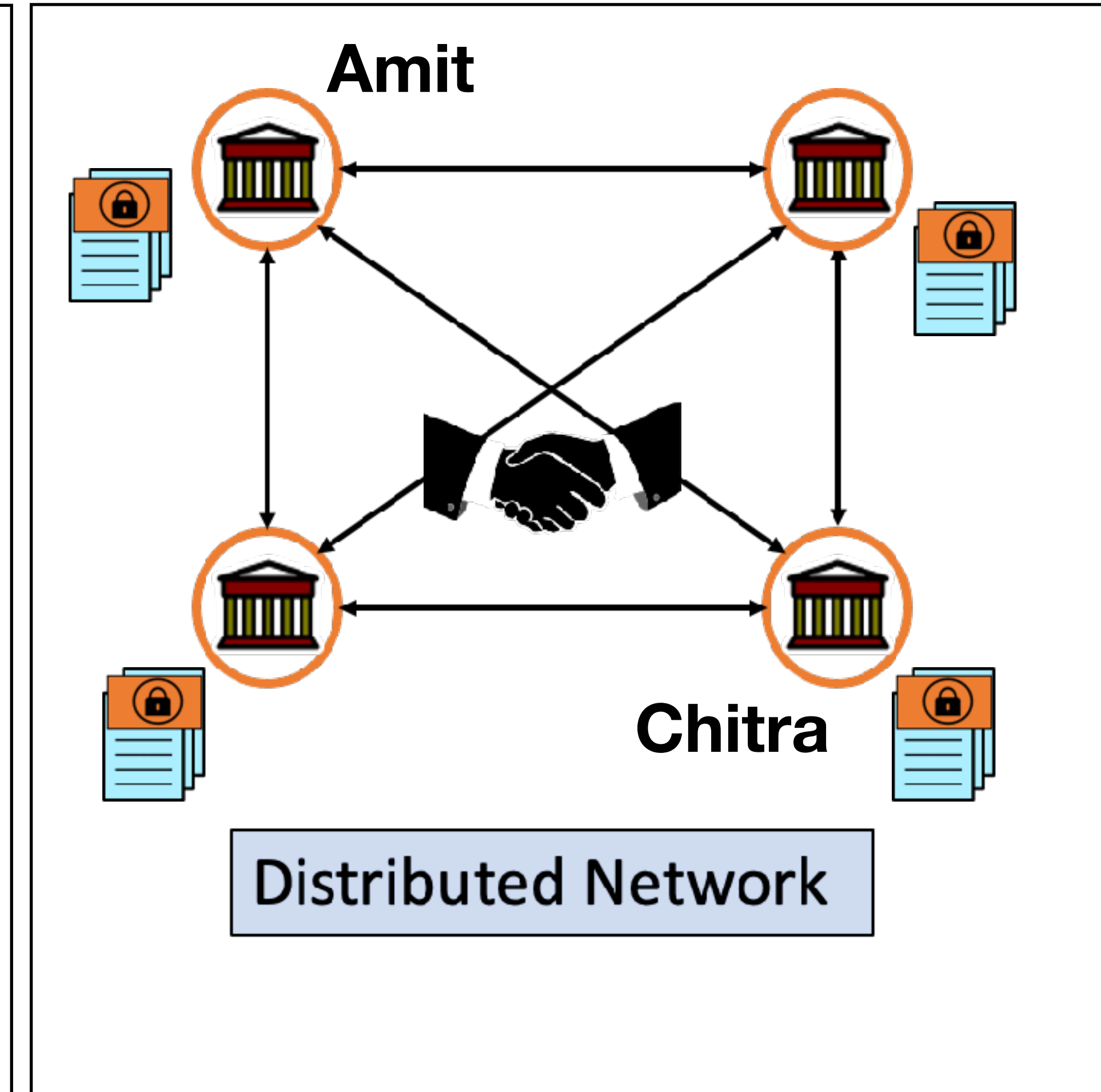
Exchange of Value: Digitally?

- Digital cash: **Big problem**
 - Can be copied easily — **Double spending!**
 - **Solutions** - requires a **SINGLE** consistent ledger
- **E-Cash:** By David Chaum (Circa 1985)
 - Led the foundations for **private digital payments** using **Blind Signatures**
 - Not a decentralised solution — Bank is the **issuer** and **double-spending gatekeeper**
 -

Blockchain Basics



Centralised Set-up

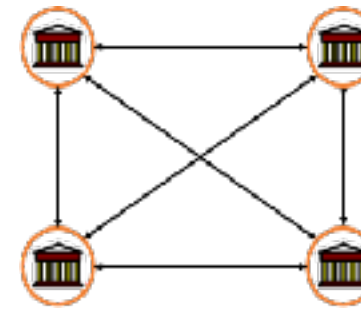


DeCentralised Set-up

Analysing the Benefits of Decentralisation

Properties

No central authority



Decentralisation gives
Self-sovereignty

Replicated ledger



Replication gives transparency

Updates via consensus

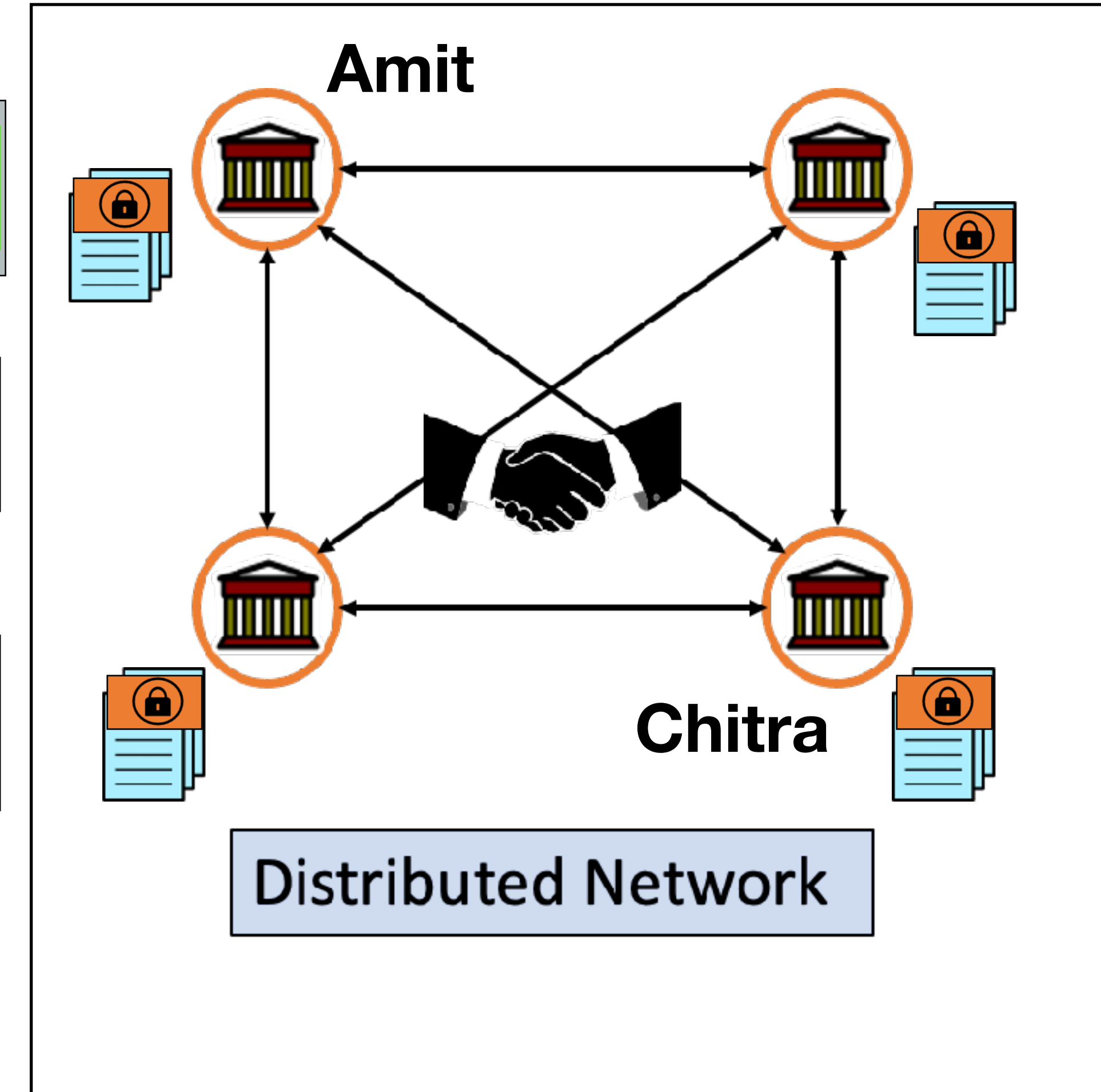


Consensus gives trust and
reliability

Immutable updates to ledger

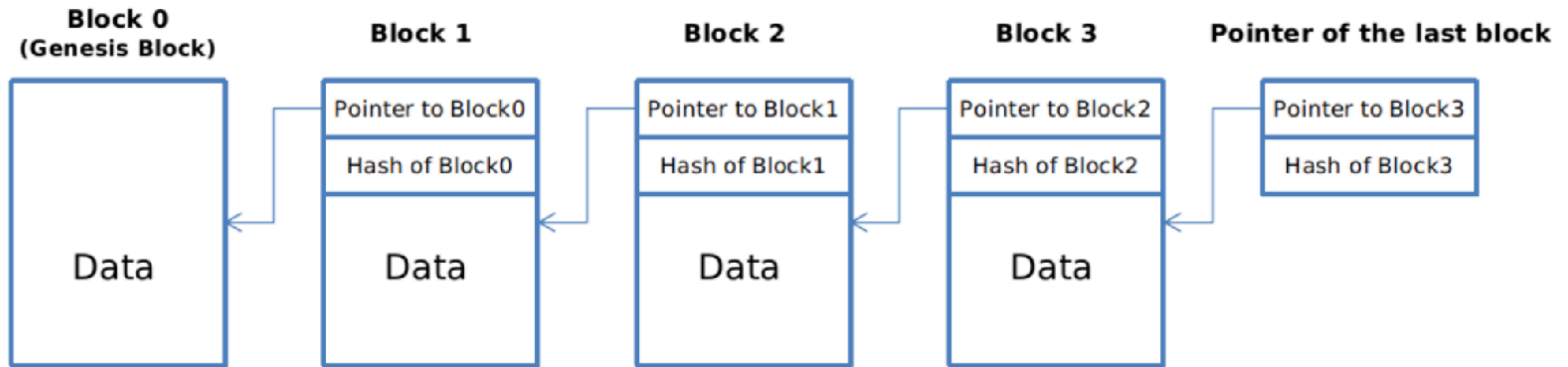


Permanence gives integrity
Traceability



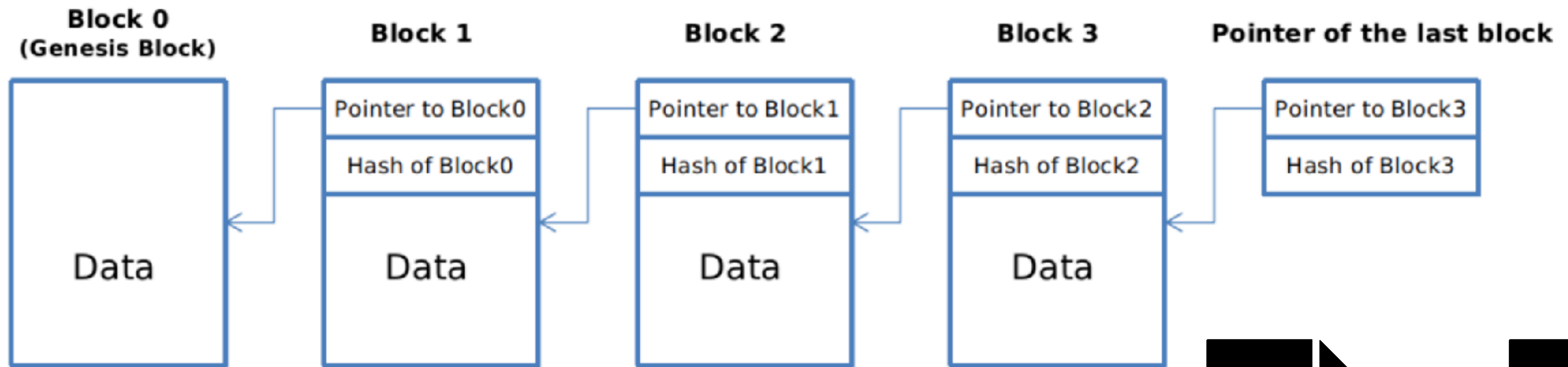
DeCentralised Set-up

Blockchain — A 5000 Feet View!

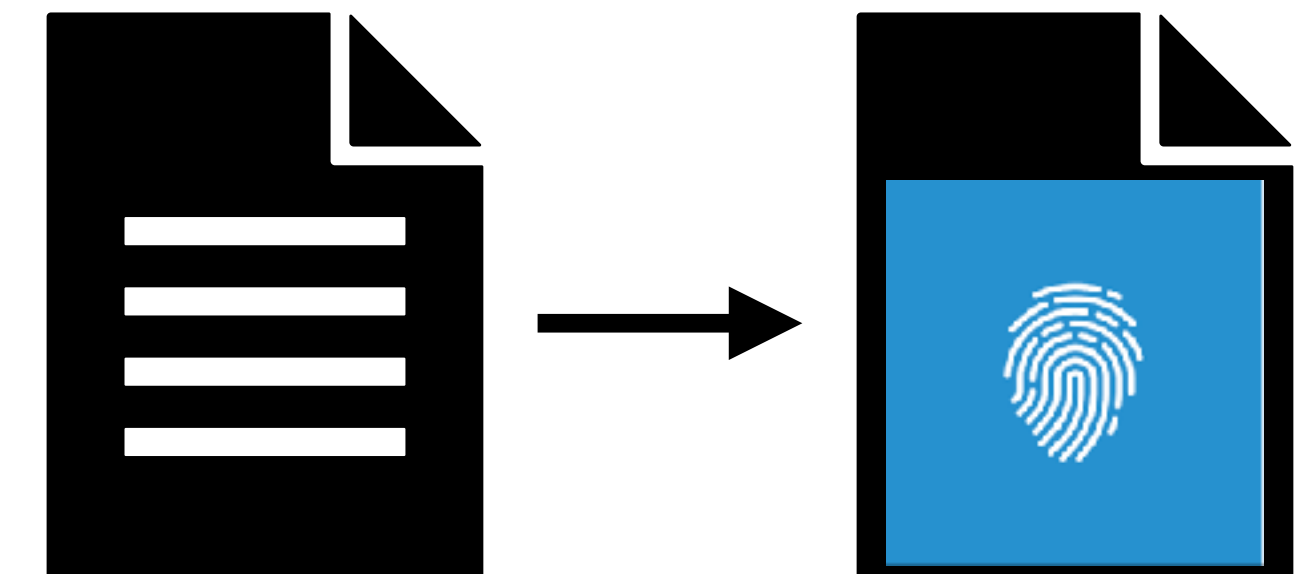


- A group of transactions are stored as **block** of data
 - Data can also include contracts — called **smart contracts**
- The blocks are **chained** together using cryptography!
 - To ensure that no one can tamper with the **committed data**

Blockchain — A 5000 Feet View!

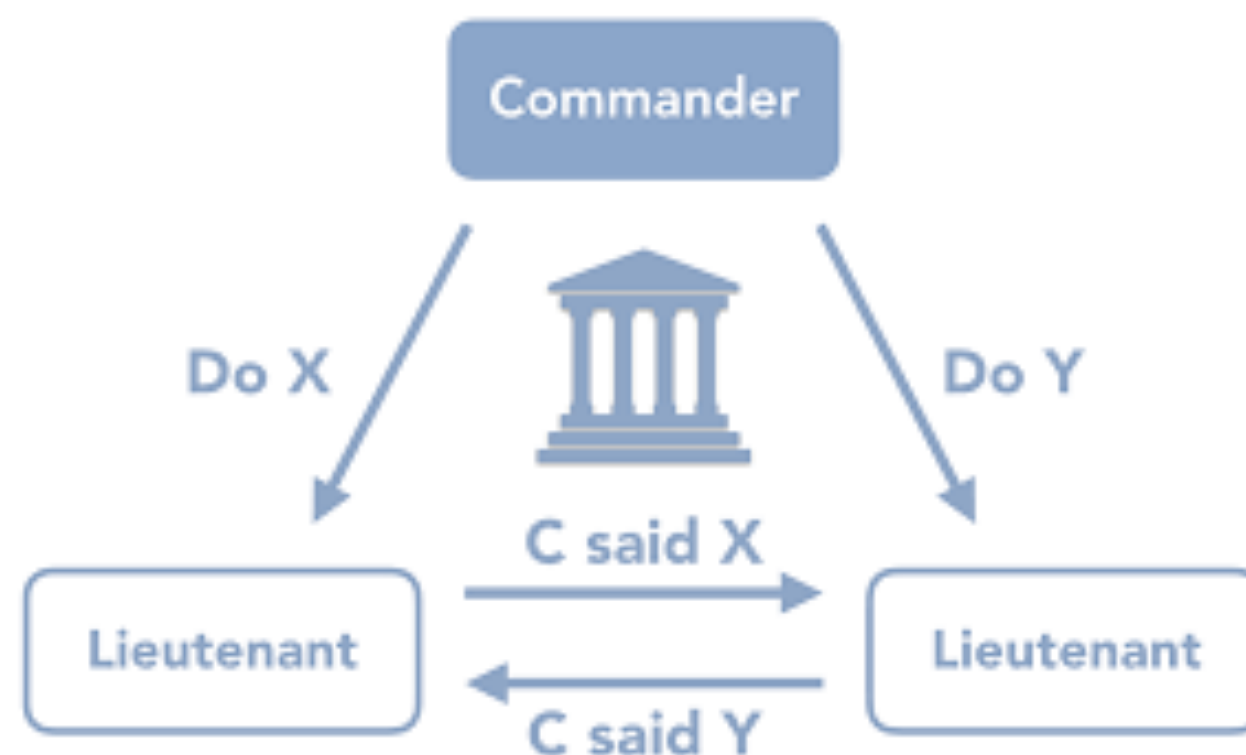
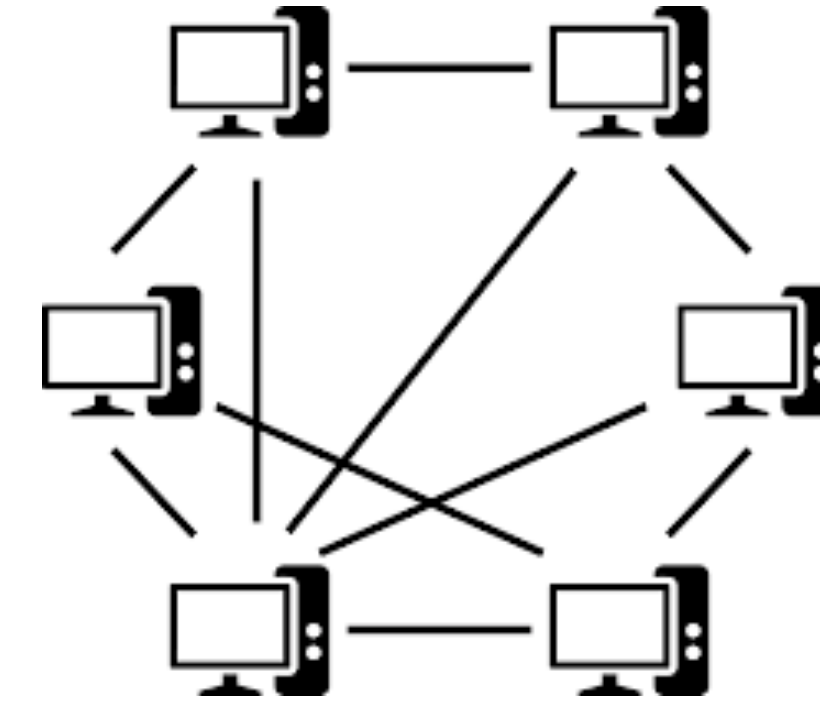


- A group of transactions are stored as **block** of data
 - Data can also include contracts — called **smart contracts**
- The blocks are **chained** together using cryptography!
 - To ensure that no one can tamper with the **committed data**



Key Components of Blockchain

- Peer-to-peer Network — Decentralisation (thus reliability)
- Digital ledgers (possibly replicated)
- Cryptography — Immutability, Integrity
- Consensus — Transparency and Trust



Goal: Decentralisation!



In Search of a Trust Protocol

From Credit to Crypto: Bitcoin was not the first attempt!

- Bitcoin doesn't provide the same level of anonymity as cash, but close enough.
- Earliest ideas of applying crypto to cash
 - Chaum, Fiat, Naor 1988 — **Detect double spending**
 - Chaum created a company called **DigiCash in 1989**
 - 1991: Stuart Haber and W. Scott Stornetta invented cryptographic linking of data in an **“append-only”** data structure.
 - 1993: Proof-of-work established; as a way to deter **denial-of-service attacks** and other service abuses such as **spam** on a network.
 - 2009: **Bitcoin: A peer-to-peer electronic cash** (<https://bitcoin.org/bitcoin.pdf>)

Nuts and Bolts — Cryptography



What are the requirements?

- Every object of value must have certain invariants:
 - Scarce
 - Unique (must have measures in place to detect counterfeiting)
 - Immutable
- A possible solution in the digital realm is through cryptography
 - Cryptography: Provides a mathematically rigorous mechanism by which information can be made to have the following properties:
 - Confidential
 - Integrity
 - Non-repudiability



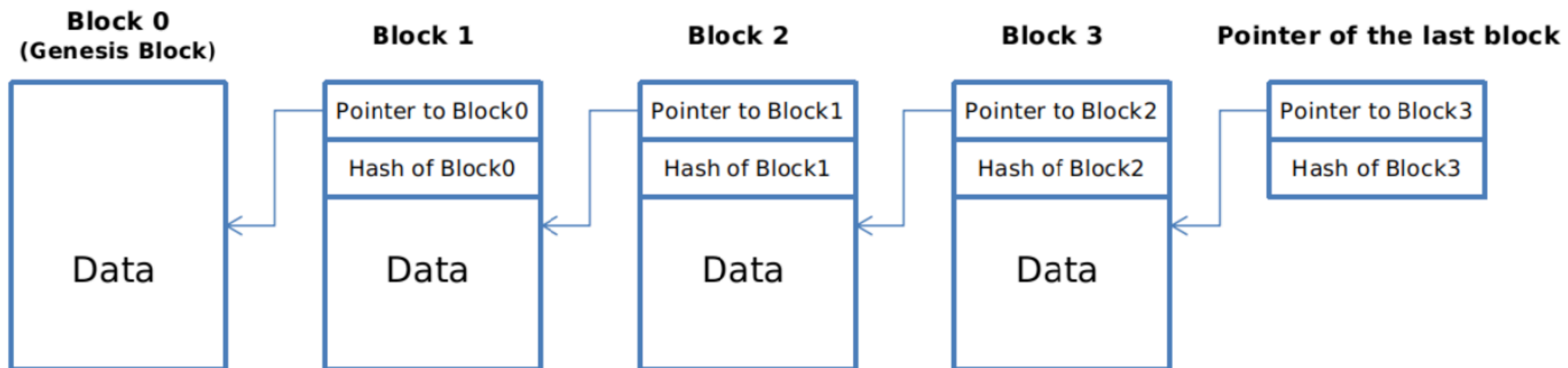
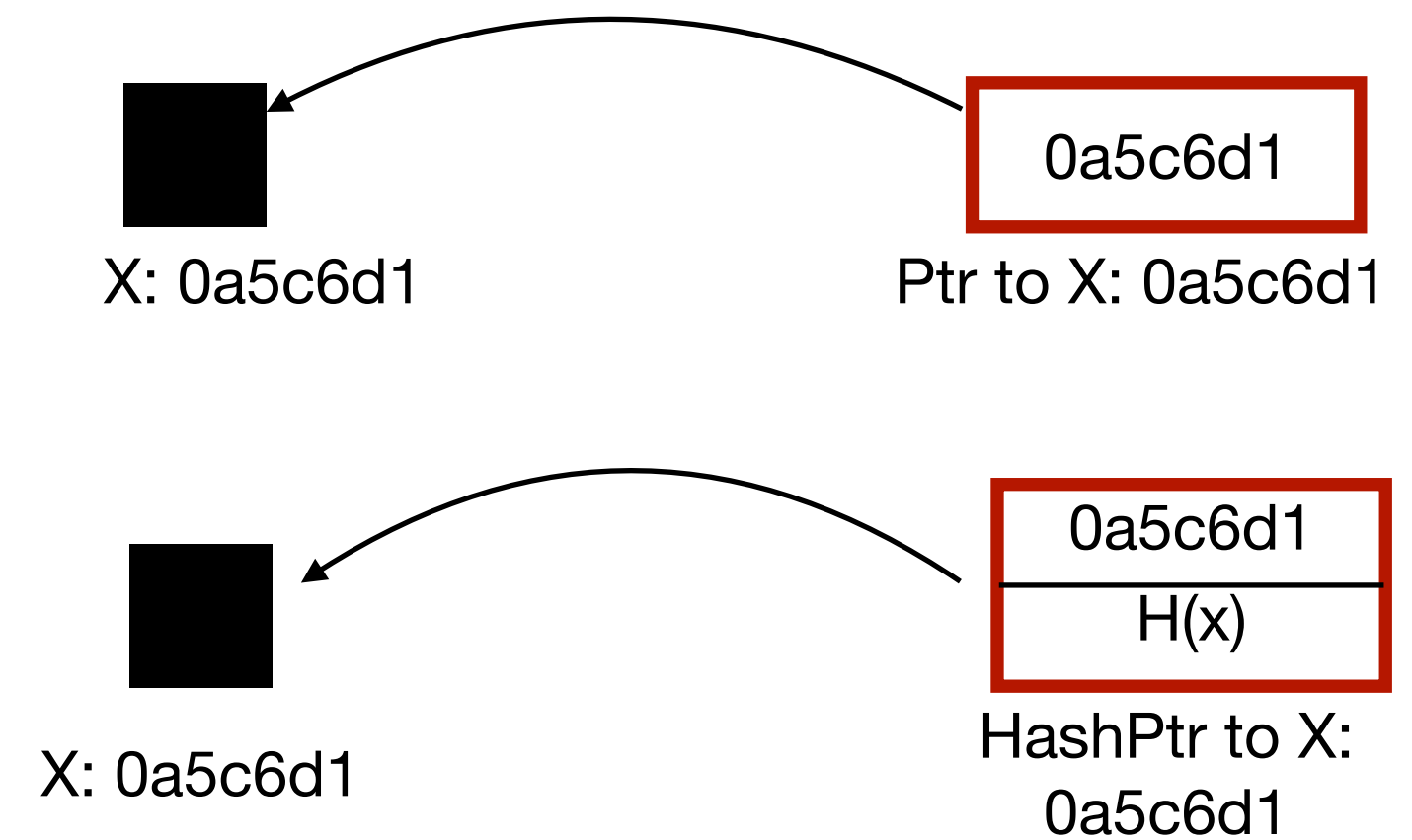
Integrity: Cryptographic Hash Functions

- A hash function $H(\cdot)$ that has the following features:
 - Takes in **arbitrary-sized** input and produces a **fixed-size** output
 - It is a **non-invertible** function. That is, given an output one cannot compute the input
 - Forward computability is eminently **efficient** — typically $O(n)$ where n is the input size
- For hash functions to be secure following invariants must be satisfied:
 - **Collision-resistance**: It is infeasible to find two values, x, y , s.t. $x \neq y$ yet $H(x) = H(y)$ with high probability
 - **Hiding**: Given y s.t. $y = H(x)$, there is no feasible way to find x
 - In fact it is $H(r \parallel x)$ where r is chosen from a distribution with high min-entropy
 - **Puzzle-friendliness**: if for every possible n -bit output value y , if k is chosen from a distribution with high min-entropy, then it is infeasible to find x such that $H(k \parallel x) = y$ in time significantly less than 2^n .

From Hashes to Hash Pointers

From Integrity to Tamper-evidence

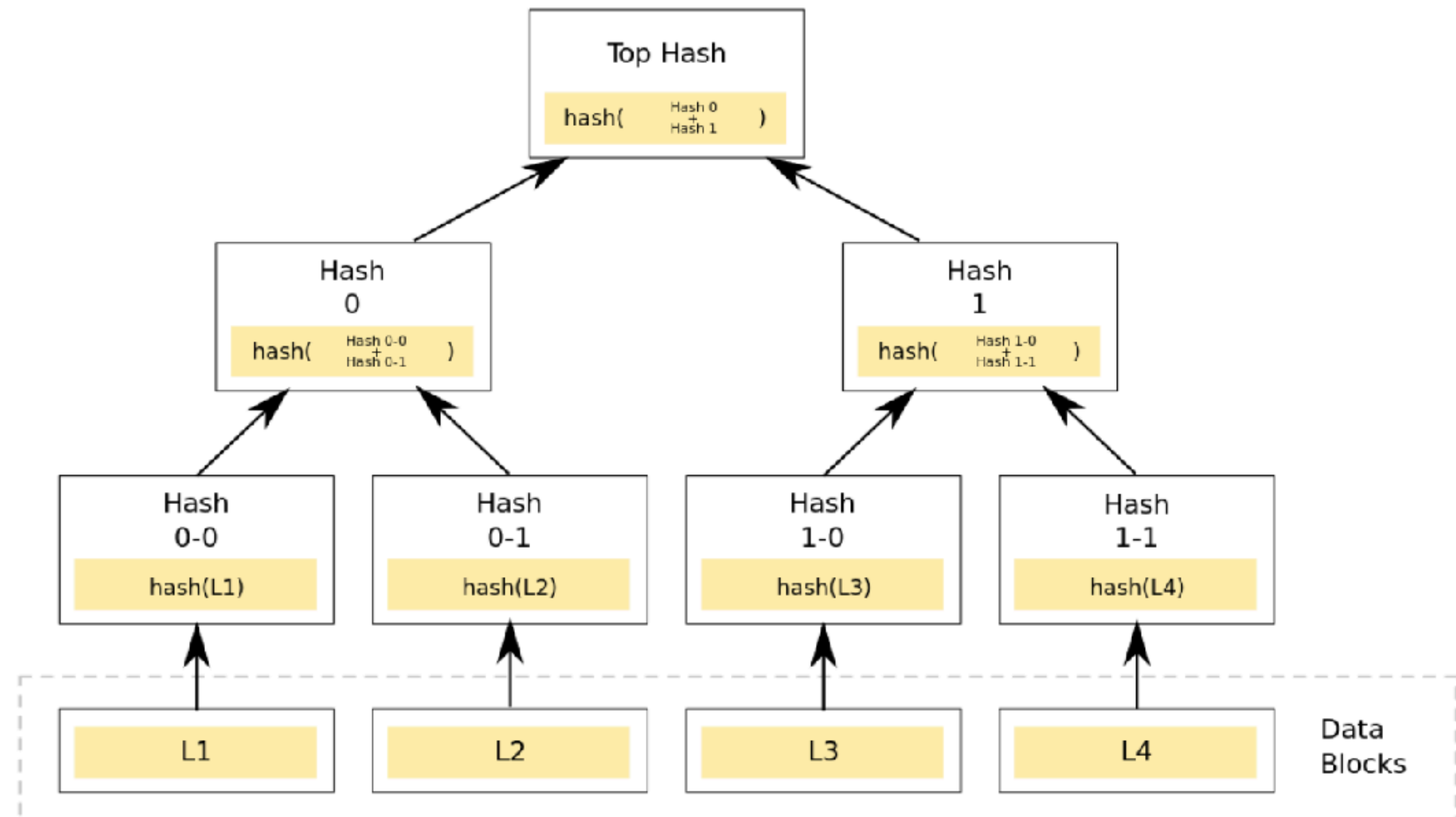
- A **pointer**: object that stores mem address
- A **hash pointer** additionally stores cryptographic hash.
- Tamper-evidence property



Merkle Trees

Moving towards efficiency

- Merkle tree is a binary tree of **hash pointers**
- Quick tamper-evidence check
- Concise proof of **membership**
- Proof of **non-membership** in log time and space



Digital Signatures

Identities and non-repudiation

- Properties:
 - Cannot be forged; Cannot be snipped off
- Public keys as identities — can make as many of them as one desires; need to be careful of **sybil attacks**

